

```
source src {
    unix-dgram("/run/systemd/journal/syslog");
    internal();
    file("/proc/kmsg");
};
```

Now, before you can do anything, you need to actually install Systemd. Run the following:

```
$: sudo pacman -Sy systemd
$: sudo pacman -Rsu sysvinit initscripts
$: sudo pacman -S systemd-sysvcompat systemd-arch-units
```

```
$: yaourt -Sb systemd-ui-git
```

You'll now have to edit one last file, `/etc/systemd/systemd-journald.conf`, and add the line below:

```
ImportKernel=no
```

This will disable `systemd-journald` from reading `/proc/kmsg`, because `syslog` is already doing it. In fact, in the future, you won't even need `syslog`, because `systemd-journald` will do everything the former does, and more.

Now cross your fingers and reboot. You might need to force the computer off manually, because we've removed `init`, and the program needed to halt the computer is gone. When you start it up again, if all goes well, you'll be back in a text terminal in no time. If not—and in my case, it was not—you might have to wait for up to five minutes. But eventually, it will come up—so if there's no hard disk activity for more than a few minutes, don't worry.

Once you're at the TTY, log in and run the following:

```
$: sudo systemctl enable NetworkManager.service
$: sudo systemctl enable gdm.service # or kdm/slim/xdm/lxdm
$: sudo systemctl enable syslog-ng.service
```

Then reboot. You'll be greeted by your graphical login manager now. Log in, and you're set!

Troubleshooting and optimising Systemd

On my first try, Systemd took 109 seconds to boot, up from 30 seconds with SysV-init. Wait a minute... what was that, again? I thought this thing was supposed to make it all faster. Well, off to troubleshooting central for me. The first line of information about Systemd is `systemd-analyze`. Run it without any arguments, and you'll get the start-up time for the current boot:

```
$: systemd-analyze
Startup finished in 6616ms (kernel) + 12945ms (userspace) = 18962ms
```

This gives the amount of time taken to boot up the

system, but not much else. To get a more detailed output, run `systemd-analyze blame`, as shown below:

```
$: systemd-analyze blame
4840ms media.mount
3616ms sys-kernel-debug.mount
3550ms udev.service
3518ms systemd-remount-api-vfs.service
3515ms dev-hugepages.mount
3499ms dev-queue.mount
~
~
9ms accounts-daemon.service
8ms proc-sys-fs-binfmt_misc.mount
5ms syslog-ng.service
5ms upower.service
3ms rtkit-daemon.service
1ms tmp.mount
```

Frankly speaking, I have no idea how to read this. It's supposed to give me the amount of time taken by each unit to start up. But remember, since units are started in parallel, the sum of the time taken by two units to start up may be more than the time taken to actually start them up. It's also plain wrong—on my system, the `systemd-tmpfiles-setup.service` unit was taking 200ms to start up (as per *blame*), but in reality, it was holding up the boot process for a full 1.5 minutes. And how does one find that out? You can generate a boot chart using `systemd-analyze`. Do:

```
$: systemd-analyze plot > /tmp/bootchart.svg
$: firefox /tmp/bootchart.svg
```

Now, isn't that pretty? It shows you the service start-up and parallelism in great detail. What you're looking for is huge differences in time between the start-up of two units. For example, in my case, `systemd-tmpfiles-setup.service` started up at the 9s mark, but the next unit started up at the 99s mark. Now that you've identified the problem, Google around and you'll find the fix. In my case, the problem was that I had an invalid `fstab` entry for the swap partition, which was causing a time-out after 90 seconds. I just fixed that, and the boot time went from 109s to 18.9s.

Using Systemd

This article is already pretty long, so I'll end this quickly. The `systemctl` tool is your friend here. Basically, here's what you do. To see a list of running units, use the following command:

```
$: systemctl list-units
```

To see a list of all available units, use:

```
#: systemctl list-units -a
```